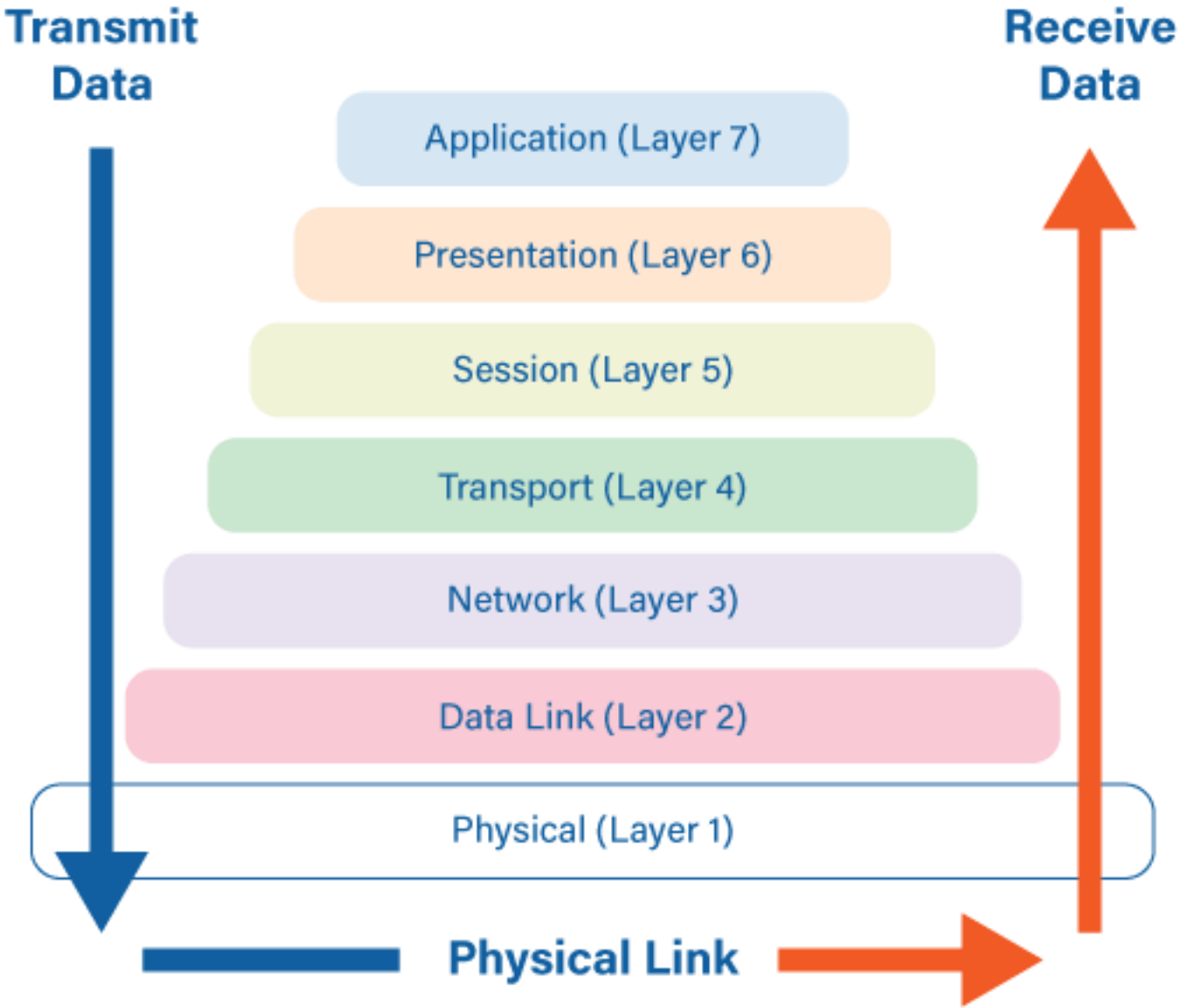
# Networking Basics

For Multiplayer Games Part 2


MCAST
Institute of Information & Communication Technology

david.deguara@mcast.edu.mt

# The 7 Layers of OSI



# OSI (Open Systems Interconnection) model

**1. Physical Layer**

- The actual hardware - cables, Wi-Fi signals, network cards. As a game dev, you rarely touch this directly, but it affects latency and bandwidth.

**2. Data Link Layer**

- Handles communication between devices on the same local network (MAC addresses, Ethernet). Mostly invisible to you, but LAN games operate here.

**3. Network Layer**

- IP addressing and routing - this is where packets find their way across the internet. When your game connects to a server at an IP address, you're working at this layer.

# OSI (Open Systems Interconnection) model

**4. Transport Layer**

- TCP and UDP protocols - crucial for game networking:
- TCP: Reliable, ordered delivery (good for chat, login, critical data)
- UDP: Fast, unreliable (perfect for real-time gameplay where speed > accuracy)

**5. Session Layer**

- Manages connections between applications - establishing, maintaining, and terminating sessions. Game matchmaking and lobby systems conceptually work here.

# OSI (Open Systems Interconnection) model

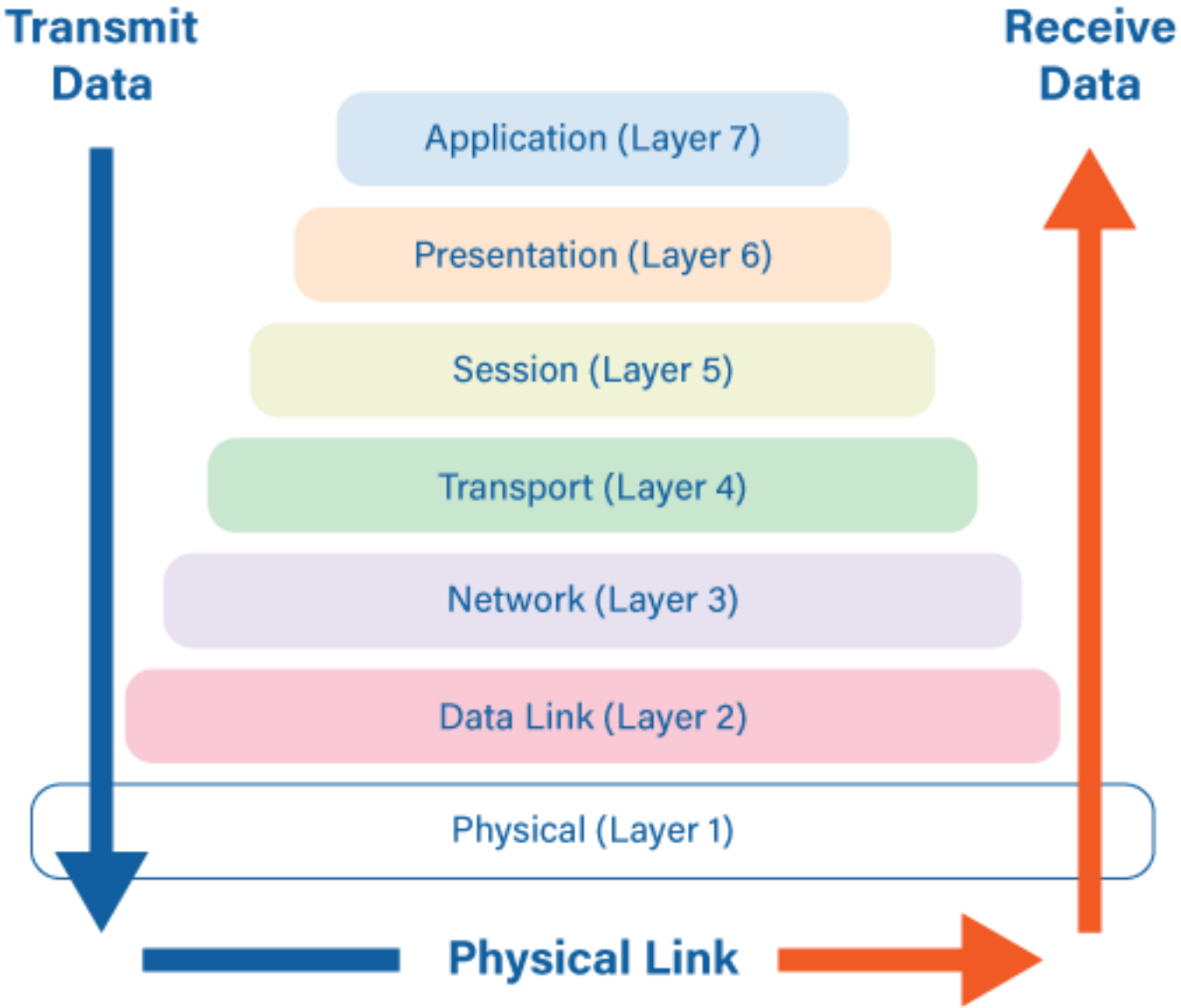
**6. Presentation Layer**

Data formatting, encryption, compression. When you serialize game state into JSON or binary formats, or encrypt player data, you're working at this layer.

**7. Application Layer**

Your game code lives here! This is where HTTP/HTTPS, WebSockets, and custom game protocols operate. Your networking API calls, REST requests to servers, and game-specific protocols all happen at this layer.

# The 7 Layers of OSI



# TCP vs UDP

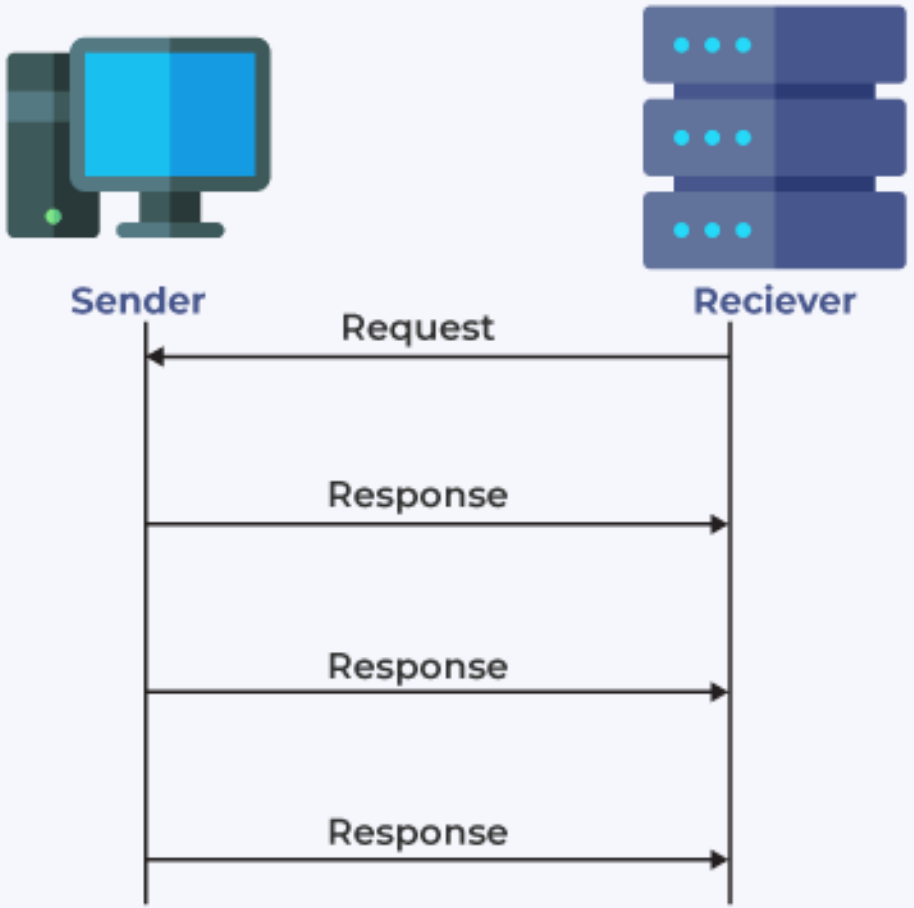
# TCP (Transmission Control Protocol)

- Connection-oriented - establishes a connection before sending data (like a phone call)
- Reliable delivery - guarantees all packets arrive and in the correct order
- Error checking - automatically detects and retransmits lost/corrupted packets
- Slower - overhead from acknowledgments and retransmissions adds latency
- Flow control - adjusts transmission speed based on network conditions
- Handshake required - three-way handshake to establish connection (SYN, SYN-ACK, ACK)

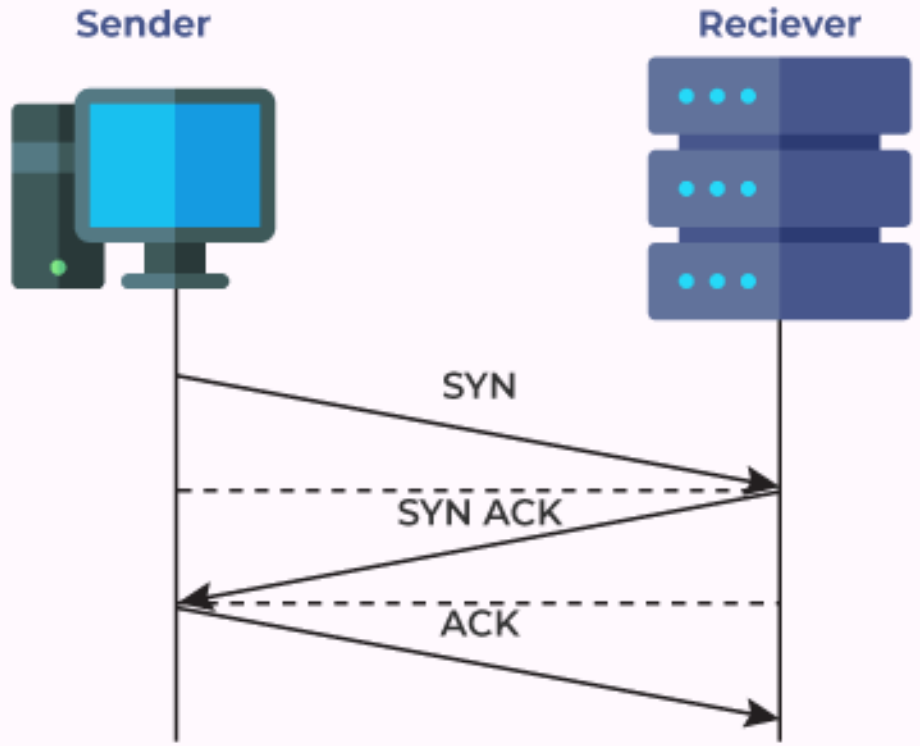
# UDP (User Datagram Protocol)

- Connectionless - just sends packets without establishing a connection (like mailing letters)
- Unreliable - no guarantee packets arrive or arrive in order
- No error checking - if a packet is lost, it's just gone
- Faster - minimal overhead means lower latency
- No flow control - sends data at whatever rate you specify
- Fire and forget - no acknowledgments or retransmissions

# UDP


Sender
Reciever
Request
Response
Response
Response

# TCP


Sender
Reciever
SYN
SYN ACK
ACK

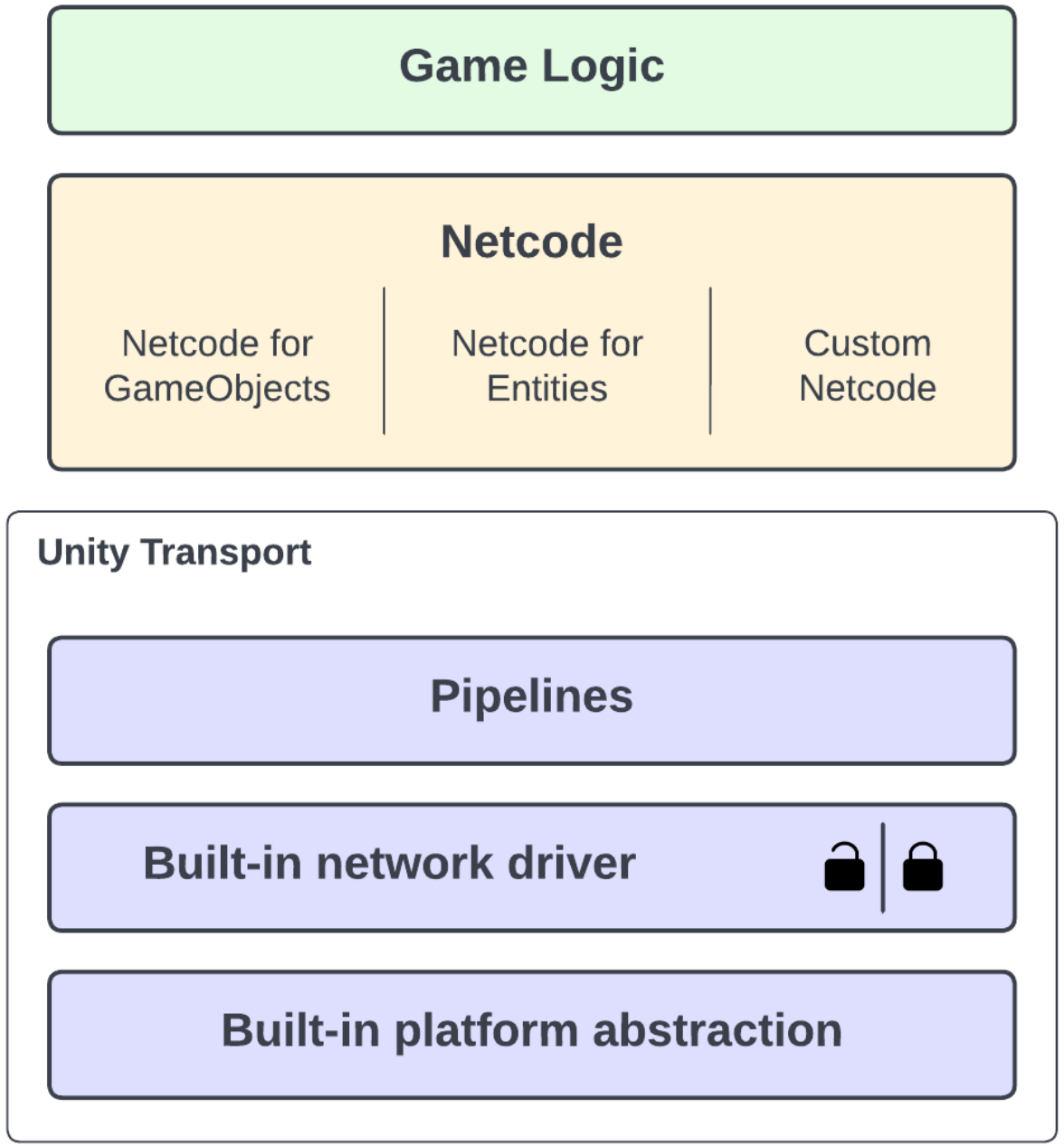
Protocol Choice

# Why might you choose UDP over TCP for a fast-paced multiplayer game?

# UTP (Unity Transport Protocol)

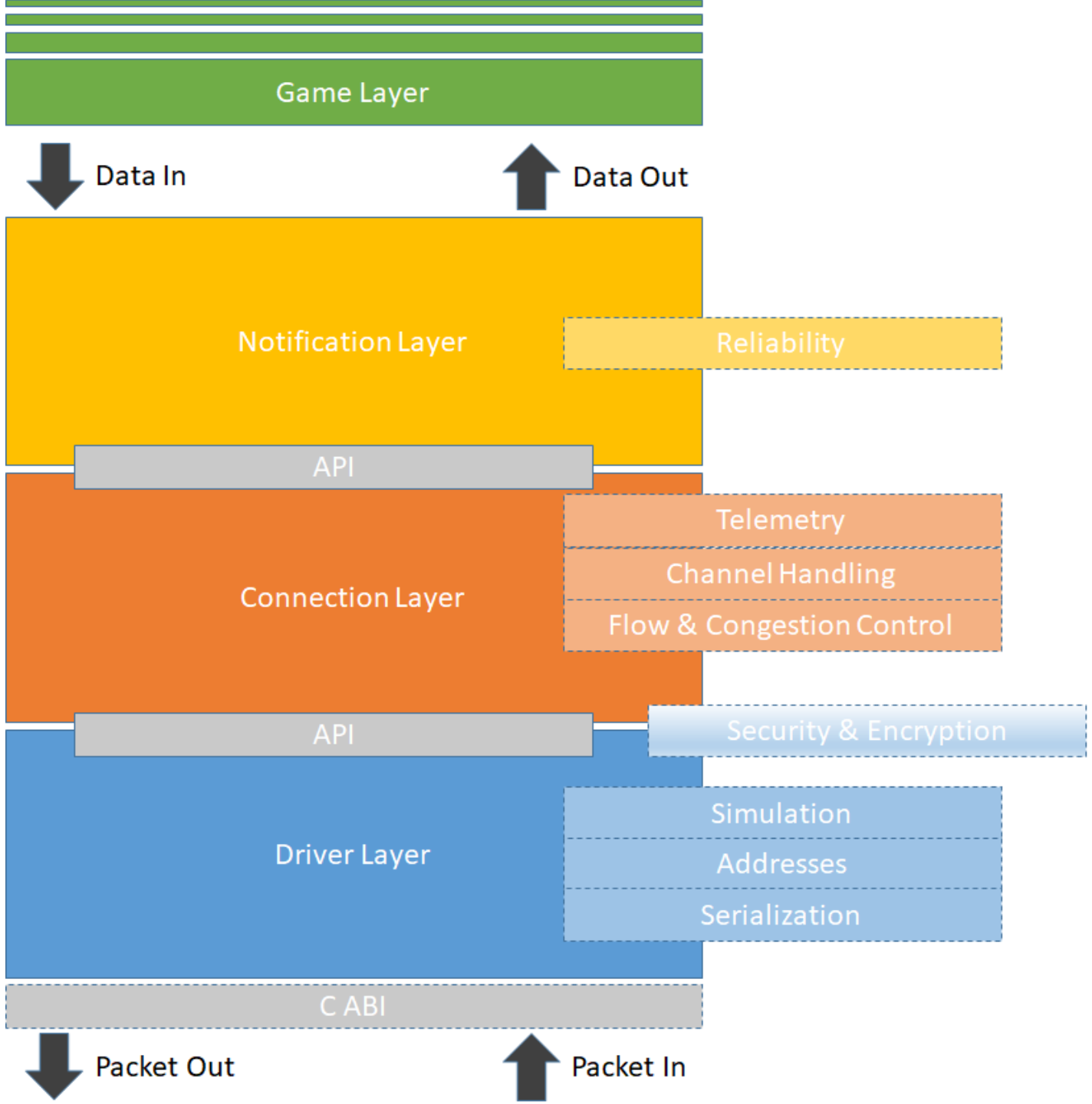
# Unity Transport Layer

- Unity Transport Package implements a sophisticated UDP-based protocol that combines the speed of raw UDP with selective TCP-like features (Similar to existing QUIC or SCTP protocols).
- High-performance networking library for Unity multiplayer games
- Designed for low-level networking with high customisability



# UTP Features

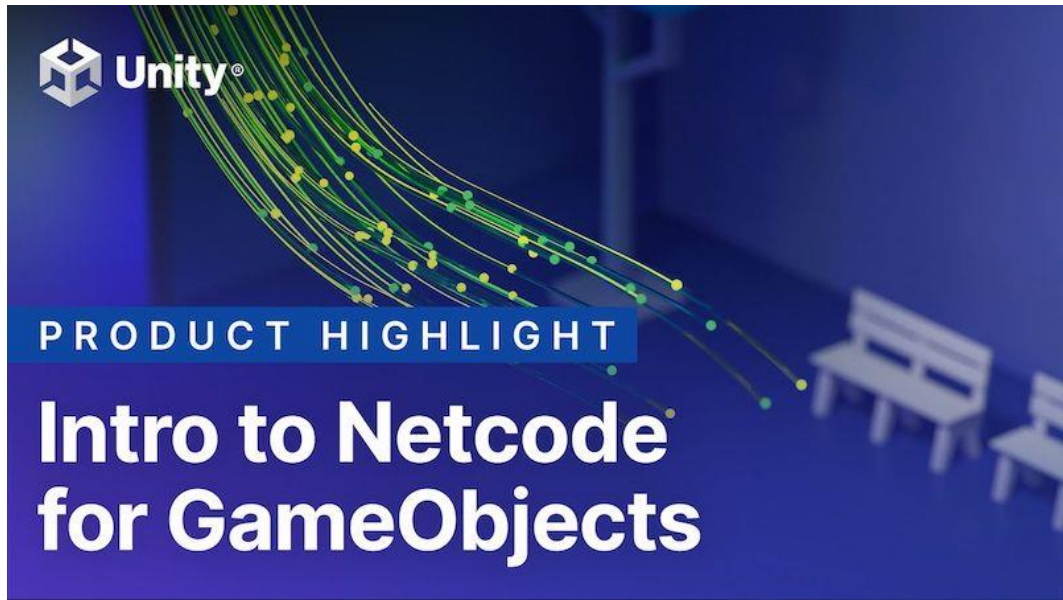
- Reliable and unreliable communication modes: Control over when packet delivery guarantees are necessary
- Fragmentation and reassembly: Handles large data packets efficiently
- Connection-based and connectionless modes: Adaptable for different network designs



# Unity Netcode

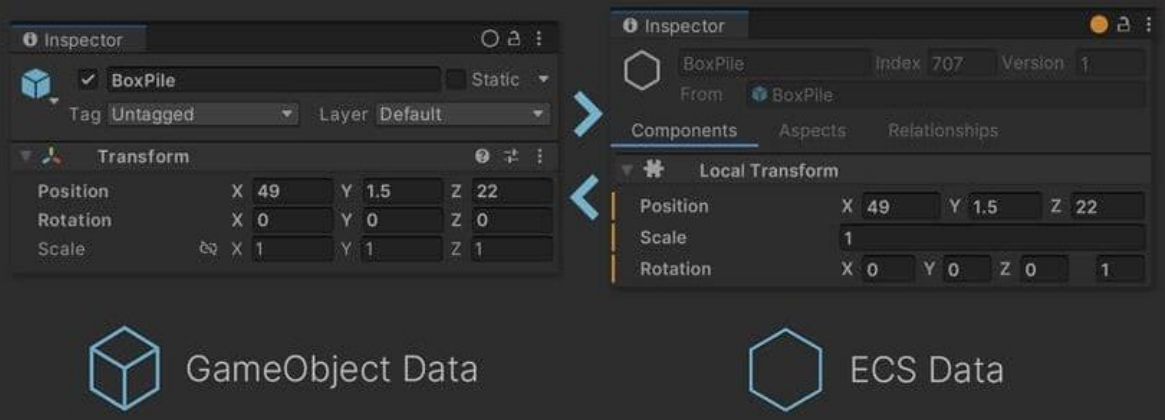
# Netcode for GameObjects

- Built-in multiplayer framework
- Simplifies networked gameplay with GameObjects
- Supports state synchronisation, RPCs, and scene management
- Provides authority models (client, server, or shared authority) for different game designs



# Netcode for Entities

- Netcode for Entities is built on the same Unity Transport Package foundation
- Optimised for high-performance, large-scale multiplayer
- Designed for ECS architecture and burst compilation




Core Keeper

# Netcode Architecture & Structure

| Aspect | Game Objects | Entities |
|---|---|---|
| **Definition** | High-level, feature-rich classes with behavior, components, and logic (e.g., Player, Vehicle, Weapon) | Lightweight data structures focused on identity and state, often used in ECS (Entity Component System) |
| **Architecture** | Object-oriented (OOP) - inheritance hierarchies, encapsulated methods | Data-oriented (DOD/ECS) - components attached to entity IDs, systems process data |
| **Memory Layout** | Scattered in memory due to polymorphism and references | Contiguous memory arrays for cache-friendly processing |
| **Ownership** | Object owns its network authority and logic | Authority managed separately from entity; systems handle logic |
| **Spawning** | Instantiate prefabs/templates, often heavy initialization | Create entity ID + attach components, very lightweight |
| **Examples** | Unity's NetworkBehaviour, Unreal's Actor replication | Unity DOTS NetCode, Bevy's networking, custom ECS solutions |
| **Learning Curve** | Easier - familiar OOP patterns | Steeper - requires understanding ECS paradigm |

# Netcode Network Performance & Behaviour

| Aspect | Game Objects | Entities |
|---|---|---|
| **Replication** | Replicate entire object state or marked properties/RPCs | Replicate component data; only sync changed components |
| **Synchronization** | Object-level sync - serialize/deserialize the whole object or specific properties | Component-level sync - only replicate dirty components per entity |
| **Prediction** | Client-side prediction coded into object methods | Prediction systems process entity components uniformly |
| **Interpolation** | Per-object interpolation logic in Update() methods | Centralized interpolation system processes all movement components |
| **Bandwidth** | Can send redundant data if not carefully optimized | Efficient delta compression; only changed component data sent |
| **Performance** | Can be slower with many objects due to virtual calls and scattered data | Highly optimized for large entity counts (thousands+) |
| **Flexibility** | Easy to add custom behavior per object type | Behavior defined by component composition and systems |
| **Best For** | Traditional games, moderate entity counts, complex behaviors | MMOs, battle royales, simulations with thousands of networked entities |

# Network Bandwidth

# Network Bandwidth Analogy

Bandwidth is like a highway..

- Bandwidth (Mbps) = Number of lanes on the highway
- Data packets = Cars traveling on the highway
- Latency (ping) = How long it takes a car to reach its destination
- Throughput = How many cars actually make it through in a given time

# More Lanes = More Capacity

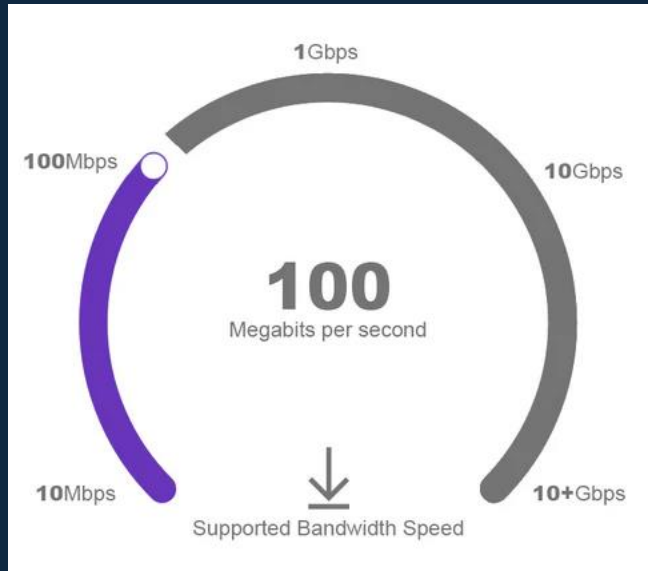
- A 1-lane road (low bandwidth) can only handle a few cars at once - traffic jams happen easily
- An 8-lane highway (high bandwidth) can handle many cars simultaneously - smoother flow

# Important: Width ≠ Speed!

- A wide 8-lane highway with a speed limit of 30 kmph (high bandwidth, high latency) gets cars there slowly
- A narrow 2-lane road with a speed limit of 200 kmph (low bandwidth, low latency) gets individual cars there fast
- For gaming: You need FAST lanes (low latency), not just MANY lanes (high bandwidth)
- For downloading games: You need a wide bandwidth.

# Definition and Relevance

- **Network bandwidth** refers to the maximum data transfer rate that a network can handle, usually measured in bits per second (bps).
- In practice, bandwidth is often described in **megabits per second (Mbps)** or **gigabits per second (Gbps)**, which determines how much data can be transmitted at a given time.



# Gigabit vs Gigabyte

- **Gigabit (Gb) = measurement of speed (how fast data moves)**
- **Gigabyte (GB) = measurement of size (how much data you have)**

**To convert download speed to actual transfer rate:**

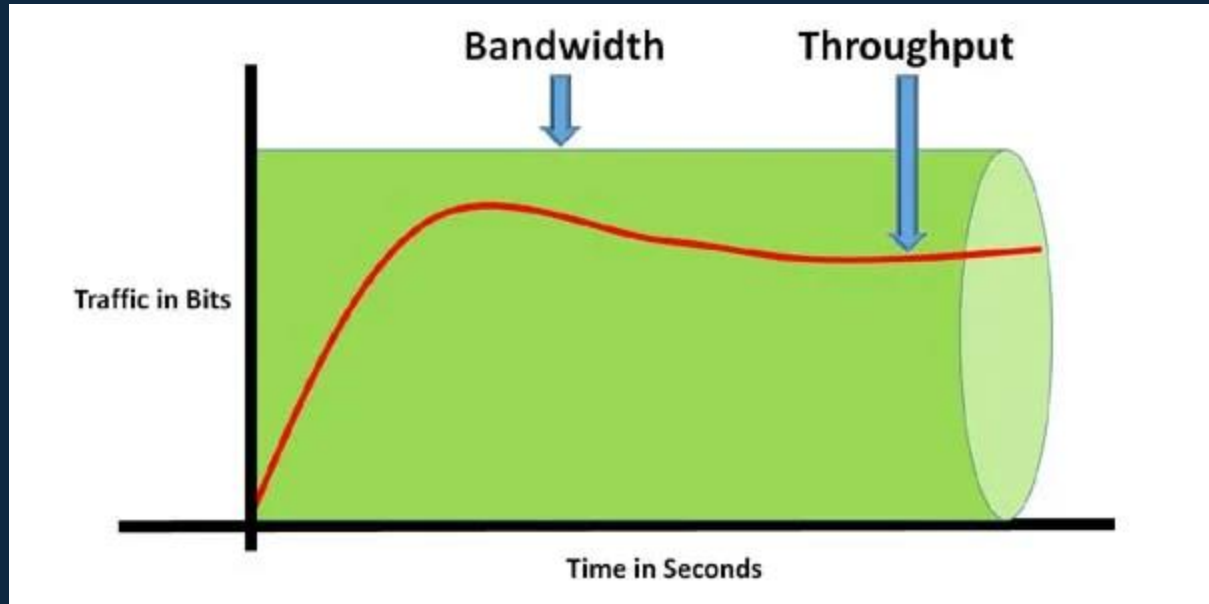
- Internet speed in Mbps ÷ **8** = Download speed in MB/s
- Example: 800 Mbps ÷ 8 = **100 MB/s** actual download speed

# Bits vs Bytes

Why use bits for speed?

# Bandwidth in Game Networking

- Bandwidth impacts how quickly game data (e.g., player movements, actions) can be sent between clients and servers.
- **Throughput** is the actual data transfer rate, which may be lower than the maximum bandwidth due to network congestion or other limitations.



# Bandwidth Hogging

# What is Bandwidth Hogging?

- Bandwidth hogging occurs when an application or device consumes an excessive amount of network resources, reducing available bandwidth for other activities.
- Common causes include streaming high-resolution videos, downloading large files, or poorly configured network devices.



# How to Manage Bandwidth Hogging

- Implement **Quality of Service (QoS)** settings to prioritise game traffic over non-essential data.
- Monitor network usage to identify bandwidth-hogging applications or devices and take steps to limit their impact, such as restricting bandwidth or upgrading the network infrastructure.

# Definition and Causes

- Lag is the delay between a player's action and the game's response.
- Lag normally occurs due to high latency, which refers to delays in data transmission across the network.
- Indicators of lag include delayed character movements or actions on the screen.

Lag in Games

# How does lag impact the fairness and enjoyment of multiplayer games?

# Effects on Gameplay

- Lag can severely disrupt the gameplay experience, leading to frustrations, especially in fast-paced games like shooters or competitive games where timing is critical.
- A common solution to mitigate visible lag is **client-side prediction**, where the game predicts the result of a player's action before confirmation from the server. This reduces the perception of delay but can lead to **rollback** in case the server's final decision differs.

# Network Latency

# What is Latency?

- Latency is the time it takes for data to travel between two points in a network.
- Although data moves at nearly the speed of light, real-world factors like distance and network infrastructure slow it down.
- Latency can also vary based on the **type of network**—for example, fibre-optic connections generally have lower latency than copper-based networks (like DSL or coaxial).
- In gaming, latency can fluctuate due to **server load**, so choosing a well-optimised server with good capacity is key to keeping latency low.

# Latency in Game Networking

- In Unity's Netcode for GameObjects, maintaining low latency is critical for ensuring smooth, synchronised gameplay. High latency can lead to **rubber-banding**, where characters snap back to previous positions due to delayed server updates.
- Developers should account for **latency compensation** techniques, such as interpolating and extrapolating positions, to smooth out any delays in data transmission.

# What practical steps can a player take to reduce high latency in a multiplayer game?

# What practical steps can a developer take to reduce high latency in a multiplayer game?

# Reducing Latency

# Techniques to Minimise Latency

- **Content Delivery Networks (CDNs):** Cache static content closer to players to reduce load times.
- **Compression:** Compress game data for faster transmission, reducing the amount of data that needs to travel between the client and server.
- **Ethernet over WiFi:** Wired connections typically reduce latency compared to wireless ones, as WiFi is subject to interference, which can increase delays.
- **Bandwidth Optimisation:** Increase network bandwidth to handle more data efficiently. This can prevent congestion and improve overall performance.

# Practical Solutions for Game Developers

- Use **region-based servers** to minimise the physical distance between clients and servers, as data travelling across continents can introduce significant delays.
- Implement **lag compensation techniques** like **time-stamping** each event so the server can correct for any differences in client response times. These techniques are especially useful in high-latency environments where fairness is crucial.

# Why would using Ethernet be more effective than WiFi for gaming, particularly in competitive environments?

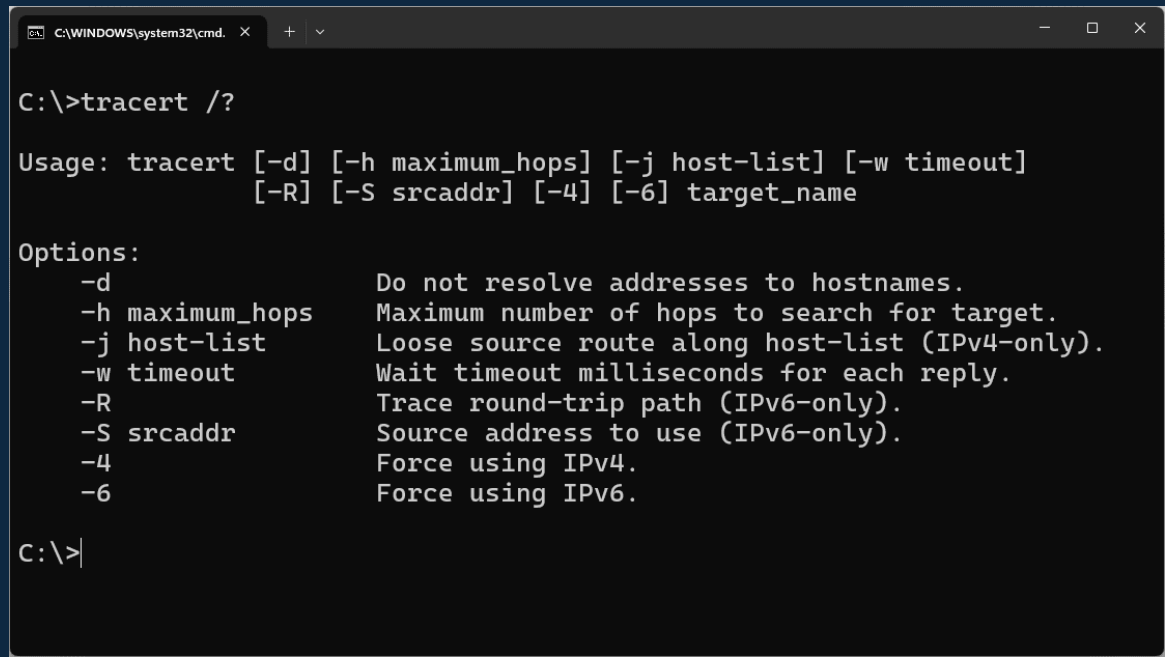
# Network Hops

# What Are Network Hops?

- Each time data is sent across the internet, it passes through several devices, such as routers, called **hops**.
- Each hop involves processing the data and forwarding it to the next device, adding a small delay.

# Importance in Multiplayer Games

- Multiple network hops can increase lag and reduce performance in online games.
- A **traceroute** command can be used to identify all the hops data takes from the client to the server, helping diagnose any issues.



```
Microsoft Windows [Version 10.0.17134.885]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\mjp>tracert ggexample.com

Tracing route to ggexample.com [96.127.135.98]
over a maximum of 30 hops:

  1     2 ms     1 ms     1 ms  LINKSYS01329 [192.168.1.1]
  2    12 ms    10 ms     9 ms  142.254.186.129
  3    32 ms    22 ms    31 ms  agg62.ycvycaam02h.socal.rr.com [76.167.16.205]
  4    14 ms    10 ms    12 ms  agg24.pldscabx02r.socal.rr.com [72.129.38.86]
  5    19 ms    18 ms    14 ms  72.129.37.2
  6    17 ms    20 ms    14 ms  bu-ether26.tustca4200w-bcr00.tbone.rr.com [66.109.3.232]
  7    18 ms    15 ms    15 ms  0.ae3.pr1.lax10.tbone.rr.com [107.14.19.56]
  8    16 ms    15 ms    14 ms  66.109.7.38
  9     *       65 ms    99 ms  ae13.cs1.lax112.us.eth.zayo.com [64.125.28.230]
 10    70 ms    85 ms    67 ms  ae6.cs1.las2.us.eth.zayo.com [64.125.27.33]
 11    66 ms    63 ms    63 ms  ae12.cs1.den5.us.zip.zayo.com [64.125.30.242]
 12     *        *        *     Request timed out.
 13    64 ms    64 ms    62 ms  ae11.er2.ord7.us.zip.zayo.com [64.125.26.251]
 14    71 ms    68 ms    66 ms  128.177.108.98.ipyx-142927-900-zyo.zip.zayo.com [128.177.108.98]
 15   105 ms   100 ms   103 ms  agg1.c13.r14.s101.chi03.singlehop.net [67.212.190.230]
 16    68 ms    73 ms    69 ms  aswg1.c25.r04.s101.chi03.singlehop.net [99.198.126.59]
 17    69 ms    67 ms    66 ms  ggexample.com [96.127.135.98]

Trace complete.
```

< Home network

< ISP

< The internet

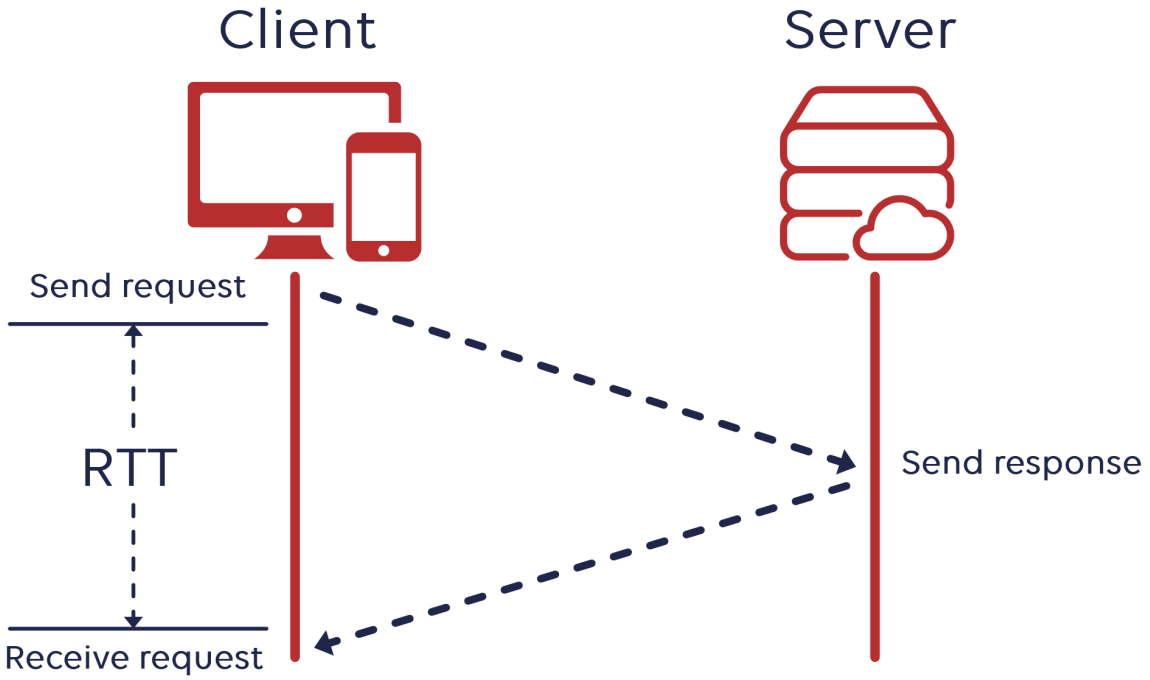
< Website host's network

< The website

# Round Trip Time (RTT)

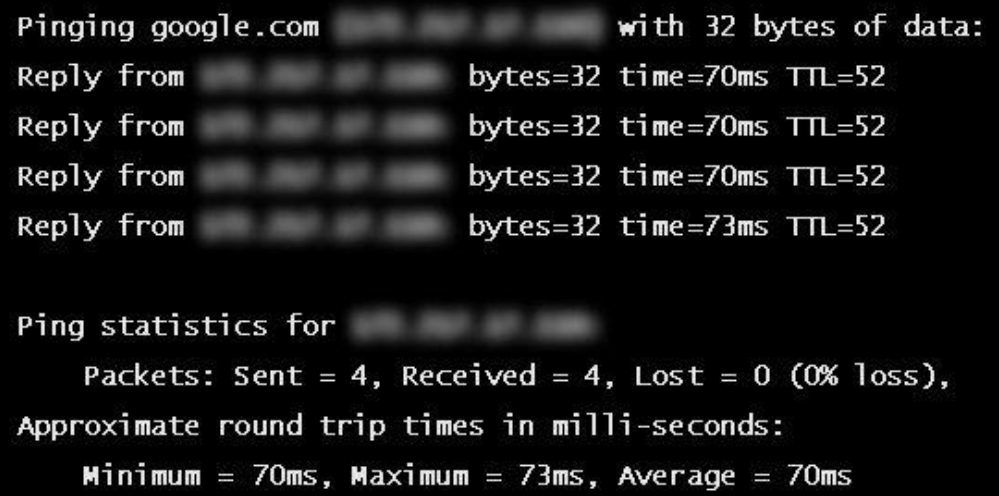
# Understanding RTT

- **Round-trip time (RTT)** measures the time it takes for a signal to travel from a client to a server and back.
- Measured using tools like **Ping**, RTT is critical in assessing network performance in games.



# Understanding RTT

- RTT is also affected by **server load**—a busy server will take longer to process requests, increasing the round-trip time.
- The **internet backbone**—the high-speed, long-distance networks that form the core of the internet—plays a key role in determining RTT, as data needs to traverse this backbone for global communication.



# Impact on Connected Games

- Lower RTT values mean faster server responses, which is essential for real-time multiplayer games.
- Unity’s Netcode can use server-authoritative architecture, where RTT plays a vital role in maintaining accurate game states. With high RTT, players may experience delayed actions or misalignment between the client and server states.


Factors Influencing RTT

# Key Determinants

- **Distance:** The greater the physical distance between the client and server, the higher the RTT.
- **Network Hops:** Each intermediary device (like routers or switches) adds processing time, which increases RTT.
- **Traffic Levels:** High traffic can cause congestion, leading to longer RTT as data waits in queues.
- **Server Performance:** A slow server with insufficient resources or too many requests will take longer to process and respond, increasing RTT.

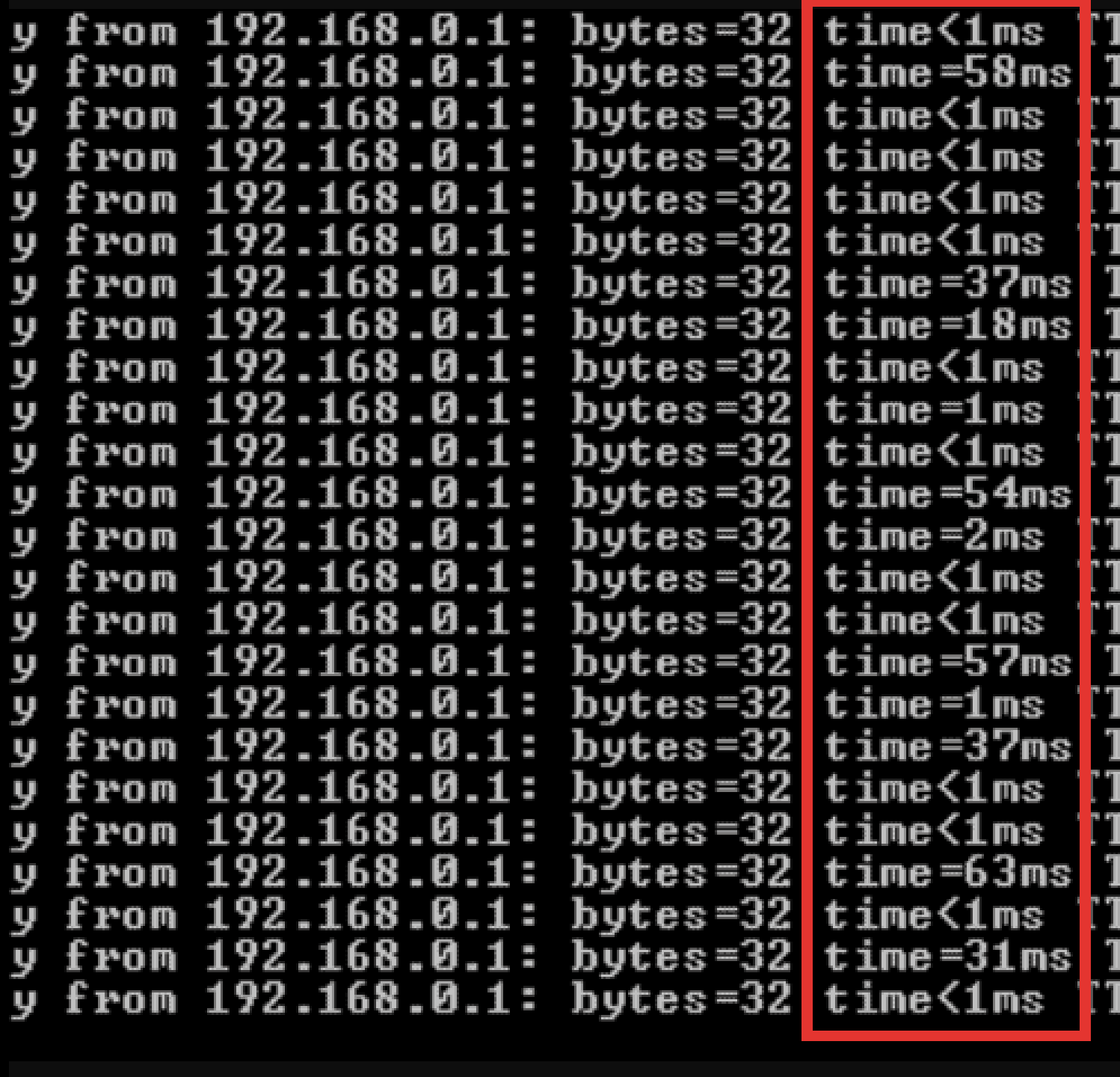
RTT and Server Performance

# How can server performance optimisation help reduce round-trip time (RTT) in multiplayer games?

# Jitter

# Jitter

- **Jitter** refers to fluctuations in packet arrival times, which can cause inconsistency in data transmission.
- In online games, jitter can result in uneven or choppy player movements, creating a poor gameplay experience.



# Definition and Impact on Games

- Jitter is often caused by **buffering** within network devices. Some routers or switches may temporarily store packets during congestion, introducing delay variability.
- Traffic prioritisation techniques, such as Quality of Service (QoS), can help reduce jitter by ensuring that game traffic is prioritised over other network traffic, like video streaming or large downloads.

## Handling Disconnections

What impact does frequent client disconnection have on multiplayer game experiences?

# Server Tick Rate

# Server Tick Rate

- Dictates the frequency at which the server processes updates and sends them to clients
  - A higher tick rate (e.g., 60 ticks per second) improves game responsiveness but requires more server resources
  - A lower tick rate reduces load but can result in delayed updates and a sluggish experience
  - Choosing the right tick rate depends on the nature of the game (e.g., fast-paced action games require higher tick rates than turn-based games)

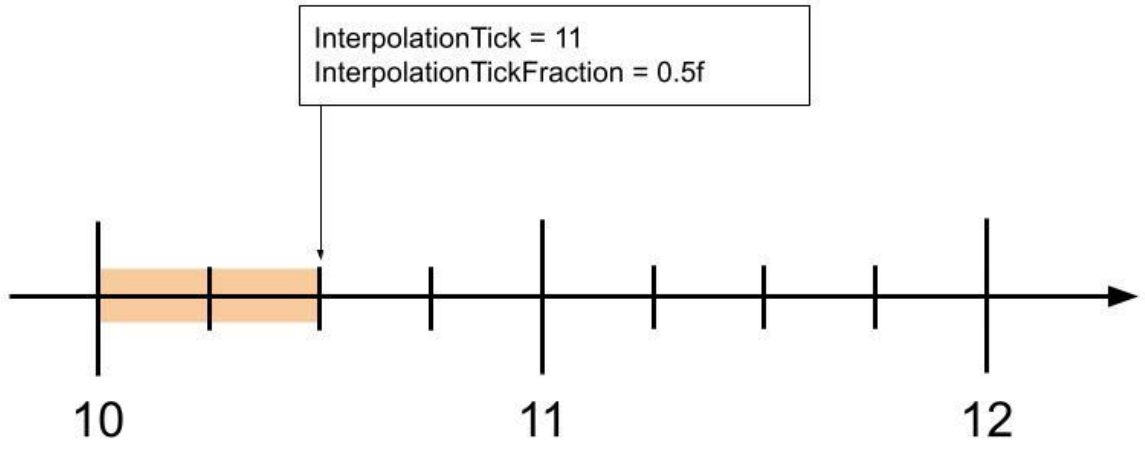
# Common Tick Rates

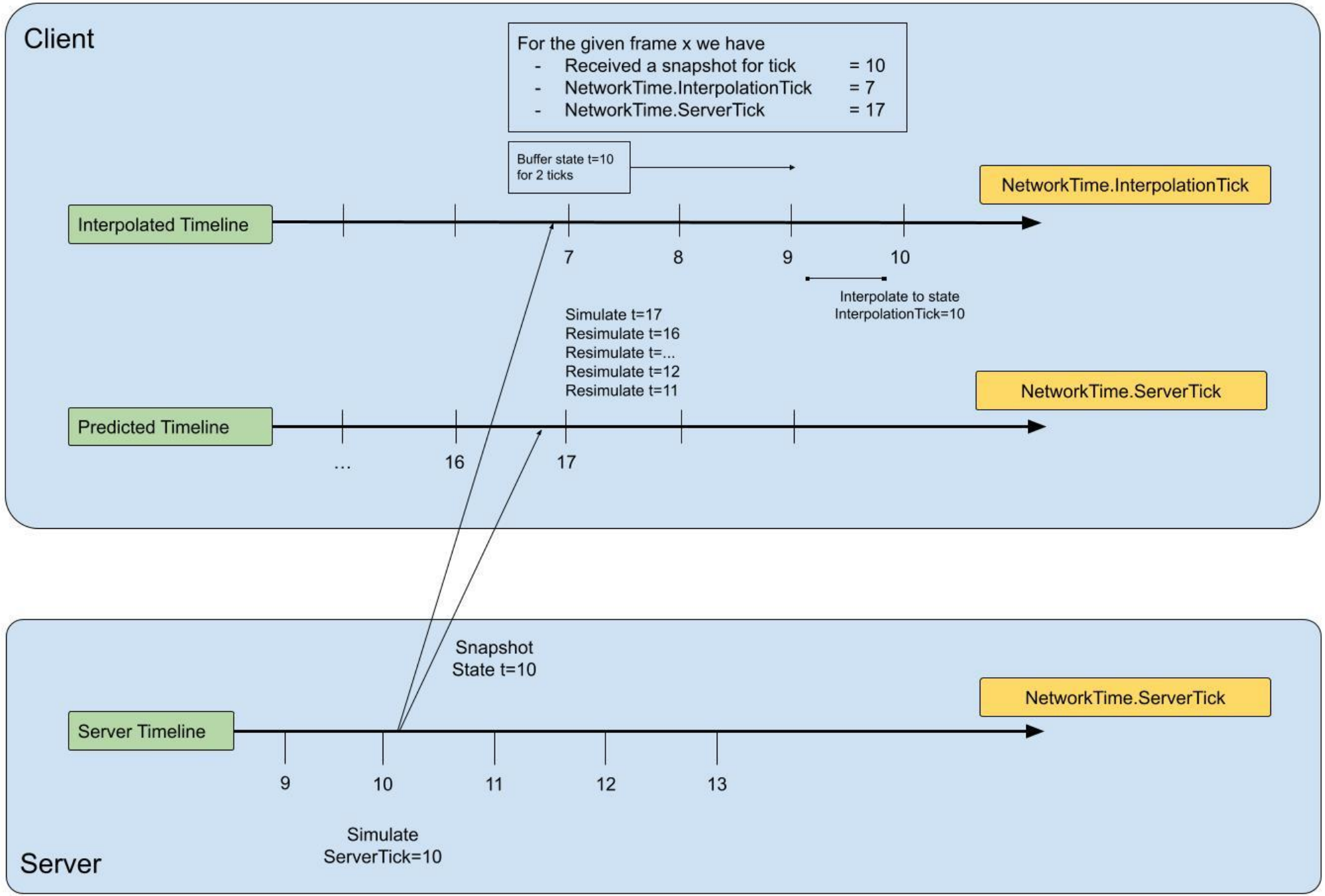
| Game | Server Tick Rate | Client Update Rate | Notes |
|---|---|---|---|
| **CS:GO** | 64Hz (MM) / 128Hz (Pro) | 64-128Hz | Competitive uses 128Hz |
| **Valorant** | 128Hz | 128Hz | Consistent high tick rate |
| **Overwatch 2** | 64Hz | 64Hz | Increased from OW1's 20Hz |
| **Fortnite** | 30Hz | 60Hz | Lower for 100 players |
| **Apex Legends** | 20Hz (BR) / 60Hz (Arena) | 60Hz | Varies by mode |
| **Call of Duty** | 20-60Hz | 60Hz | Varies by platform/mode |
| **Minecraft** | 20Hz | Unlimited | Fixed tick rate |
| **Rocket League** | 120Hz | 120Hz | Physics-heavy game |

Based on information available online (not necessarily 100% reliable)

# Interpolation and Extrapolation

- Interpolation and extrapolation are essential techniques for creating smooth gameplay between server tick updates.
- They bridge the gap between discrete server updates and the continuous motion players expect to see.




Client
For the given frame x we have
- Received a snapshot for tick = 10
- NetworkTime.InterpolationTick = 7
- NetworkTime.ServerTick = 17
Buffer state t=10
for 2 ticks
NetworkTime.InterpolationTick
Interpolated Timeline
7
8
9
10
Interpolate to state
InterpolationTick=10
Simulate t=17
Resimulate t=16
Resimulate t=...
Resimulate t=12
Resimulate t=11
NetworkTime.ServerTick
Predicted Timeline
...
16
17
Snapshot
State t=10
NetworkTime.ServerTick
Server Timeline
9
10
11
12
13
Simulate
ServerTick=10
Server

# Explanation

| Tick Value | Context | Description |
| --- | --- | --- |
| **ServerTick = 10** | Server | The authoritative time the snapshot was created. |
| **InterpolationTick = 7** | Client (Render) | The time the client is currently displaying to the player.<br>(The **Target** for interpolation will advance towards 10 and beyond). |
| **Resimulate t=11 to 17** | Client (Prediction) | The range of local input-based prediction that needs to be re-run after server reconciliation. |
| **NetworkTime.ServerTick = 17** | Client (Prediction) | The client's current predicted time, which is ahead of the authoritative server time (10). |

# Adaptive Tick Rates (Research Area)

- Dynamic Tick Rate Based on Game State
- Different areas of the game world run at different tick rates
- Lag Compensation with Variable Tick Rate
- Client-Side Prediction with Adaptive Server Tick
- Event-Driven + Fixed Tick Hybrid
- AI-Driven Tick Rate Optimization (Neural Tick Rate)

# PLEASE BE MINDFUL OF OTHERS WHO ARE USING THE LAB AFTER YOU

- Plug back in power cables that are unplugged and connect HDMI / keyboard & mouse back to PCs
- Throw away any rubbish you have brought in with you outside
- Put your chair back under the desk and keep everything orderly for the person using it after you
- Report any PCs that are not working to your lecturer

**THANK YOU**